DSP FreeRTOS 系统调试
使用指南

版本号: 1.0
发布日期: 2025.01.07

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2025.01.07	AWA1686	初始版本

广州星翼电子科技有限公司zdyzlinux

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 日志获取 **2**
2.1 串口获取 2
2.2 节点获取 2

3 栈回溯 **6**
3.1 通用寄存器 7
3.2 栈帧结构 7
3.3 执行工具自动回溯 8
3.4 计算寄存器手动回溯 10

4 指令调试 **12**
4.1 breakpoint 断点调试 12
4.2 watchpoint 观察点调试 13

5 外部 JTAG 调试接口 **15**
5.1 Xplorer 15
5.2 OpenOCD 17
5.2.1 搭建 17
5.2.2 常用命令 23

6 操作演示 **24**
6.1 OpenOCD 25

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1　tina-kernel_menuconfig 3
图 2-2　dsp-menuconfig 4
图 2-3　dsp-menuconfig 5
图 3-1　Stack Frame Layout 8
图 3-2　Stack Frame Layout 10
图 5-1　Xplorer 调试 15 16
图 5-2　Xplorer 调试 16 17
图 6-1　shared_unlock 调试代码 24
图 6-2　shared_unlock2 调试代码 24
图 6-3　崩溃现场分析示例 30

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 编写目的

本文档介绍 HiFi 系列 DSP 处理器软件调试方法。

1.2 适用范围

表 1-1: 适用产品列表

产品名称	DSP 类型
R329	HIFI4
R528	HIFI4
D1	HIFI4
T113-i	HIFI4
T527	HIFI4
R128	HIFI5

1.3 相关人员

DSP FreeRTOS 系统开发人员

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 日志获取

一般 DSP 日志有两种获取方式，通过串口实时打印日志，通过节点获取缓存日志。

2.1 串口获取

通过小机端串口实时获取日志，实时性最好。

例如：在 R128 平台上，通过串口可以看到类似日志如下：

```
---boot dsp(start_addr: 0x0d800660)---
DSP core clock frequency: 400000000Hz

 ******************************************
 **    Welcome to R128 DSP FreeRTOS     **
 ** Copyright (C) 2019-2022 AllwinnerTech **
 **                                      **
 **     starting xtensa FreeRTOS V1.0    **
 **   Date:Mar 28 2024, Time:09:39:03    **
 ******************************************

VFPU lib version:2.0.0  API version:4.31
[DSP-RPD-INF][rpdata_init](153) rpdata version:V0.5.6
[DSP-RPD-INF][_rpdata_create](452) rpd create (RVtoDSPCli,RVtoDSPCliCb)(0x0d8a04f0)
[DSP-RPD-INF][rpdata_tryconnect](570) state:0x454c4449
...

获取dsp核的内存使用情况，该打印为dsp代码打印内容：

c906>rpccli dsp free
==> Round [1] <==
Total Heap Size :  7849800 Bytes   ( 7665 KB)
        Free :  7731888 Bytes   ( 7550 KB)
    Min Free :  7727424 Bytes   ( 7546 KB)
```

2.2 节点获取

主核系统为 Linux 平台可配置通过读节点获取 DSP 日志。

Linux 内核配置，开启功能：

```
make kernel_menuconfig
Device Drivers  --->
  Remoteproc drivers  --->
    <M> SUNXI remote processor support
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[*]   SUNXI remote processor trace drivers support
```

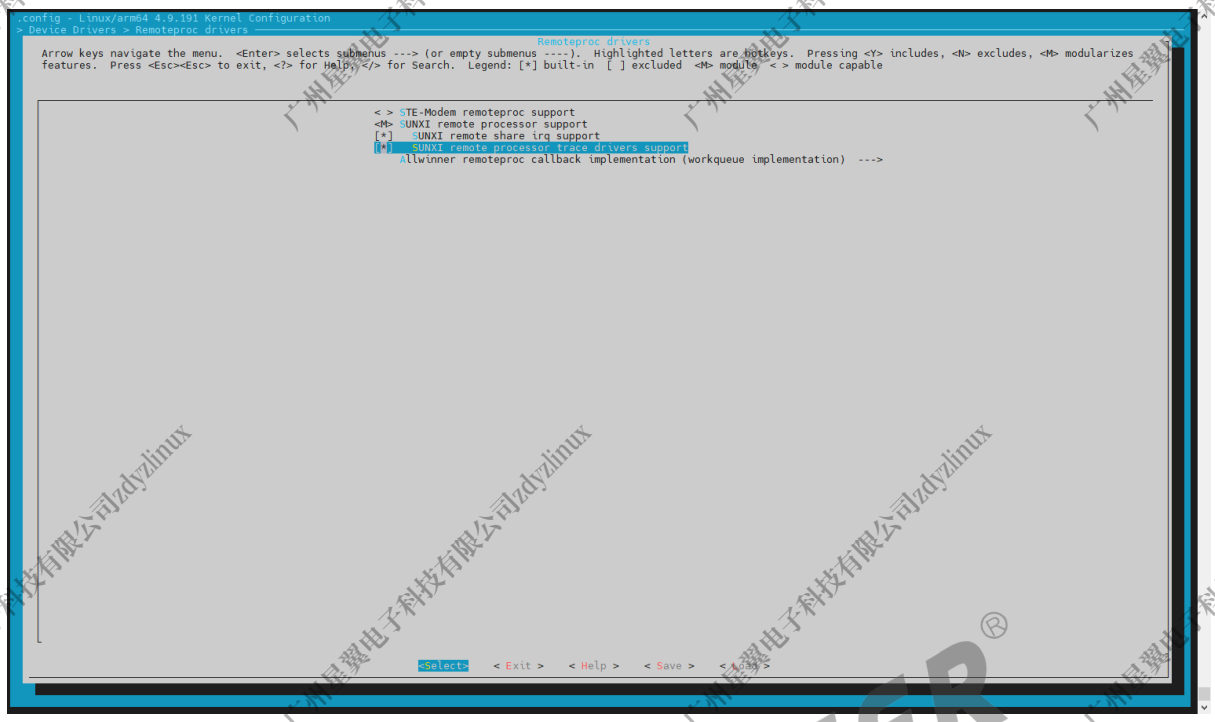

图 2-1: tina-kernel_menuconfig

DSP 配置，对应开启功能：

```
make menuconfig
Components options  --->
  Thirdparty components  --->
    OpenAMP components  --->
      [*] log trace  --->
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

图 2-2: dsp-menuconfig

说明

DSP 将打印信息放到环形 buffer 中，该 buffer 默认大小 4k。

可以通过配置来修改 buffer 大小:

```
make menuconfig

Components options  --->
  Thirdparty components  --->
    OpenAMP components  --->
      [*] log trace  --->
        --- log trace
        (0x1000) config trace buffer size
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

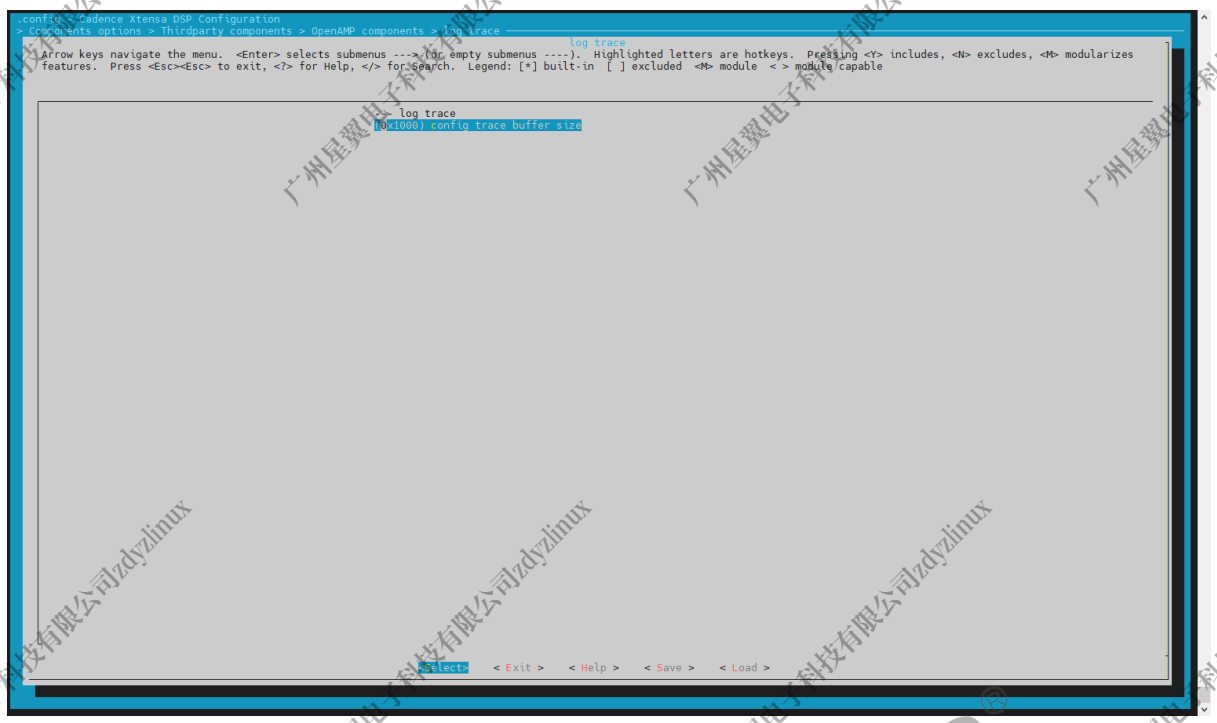

图 2-3: dsp-menuconfig

操作方法

启动 dsp 后可以 cat 以下节点查看 dsp 端的日志：

```
cat /sys/kernel/debug/remoteproc/remoteproc${dsp_id}/trace0
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 栈回溯

一般情况下，程序有 bug 触发异常，会打印出崩溃信息。包括 register dump、backtrace 和 Stack Data 三个部分；

根据这些信息可以获取系统崩溃异常原因，反推出程序的调用链信息，帮助开发者快速理清代码逻辑，提高分析问题的效率。

相关打印，例如：

```
Core 0 register dump:
PC    : 0x20202020  PS    : 0x00060133  A0    : 0xb20b2544  A1    : 0x320c9d30
A2    : 0x320c9dac  A3    : 0x32014474  A4    : 0x7fffffff  A5    : 0x320c9d50
A6    : 0xfffffff0  A7    : 0x00000002  A8    : 0xb20b8c6d  A9    : 0x320c9d10
A10   : 0x20202020  A11   : 0x320c9d30  A12   : 0x0000000c  A13   : 0x320c9dac
A14   : 0x00000004  A15   : 0x320c9ec0  SAR   : 0x0000001c  EXCCAUSE: 0x0000000c
EXCVADDR: 0x20202020  LBEG  : 0x320c16a8  LEND  : 0x320c16c4  LCOUNT : 0xffffffff

Other Core Register:
EPC   : 0x0c0321e8  DEPC   : 0x00000000  CPENABLE: 0x00000000  DEBUGCAU: 0x00000000
EPC1  : 0x0c0321e8  EPC2   : 0x00000000  EPC3   : 0x0c069ba6  EPC4   : 0x00000000
EPC5  : 0x00000000  DDR    : 0x00000000

------------
0x20202020
backtrace : invalid pc
------------

Stack Data:
 （略）
0x320c9d10: 0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5
0x320c9d20: 0xb20b191f  0x320c9d50  0x320b18c0  0x320c9e60
0x320c9d30: 0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5
0x320c9d40: 0xb2060dda  0x320c9e50  0x32106680  0x000001ff
0x320c9d50: 0x00000000  0x00000000  0xa5a5a5a5  0xa5a5a5a5
0x320c9d60: 0x320b18c0  0x320c9e60  0xa5a5a5a5  0xa5a5a5a5
0x320c9d70: 0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5
0x320c9d80: 0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5  0x00000000
0x320c9d90: 0xa5a5a5a5  0x00a5a5a5  0xa5a5a5a5  0xa5a5a5a5
0x320c9da0: 0x320c9ee0  0x320c9ec0  0x00000004  0xa5a50000
0x320c9db0: 0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5
0x320c9dc0: 0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5  0xa5a5a5a5
...
```

说明

callstack 命令依靠固件对应的 elf 文件；如后续修改过代码并重新生成了固件，elf 文件将发生改变，不能保证解析得到的栈回溯信息正确。请确保解析所用的 elf 文件与机器固件版本一致，可提前进行备份确保不会由于代码更新被刷新。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.1 通用寄存器

以下是异常发生时获取的通用寄存器和部分特殊寄存器的值。

名称	作用
PC	即触发异常时 EPC 寄存器的值，指向异常时正在执行的指令或者下一条指令
PS	Processor Status 寄存器
A0	返回寄存器的值，保存着函数跳转过程的返回地址。注意其高 31-30 bit 保存 CALLINC 窗口递增值，计算返回地址时需要或上 PC 的高 31-30 bit
A1	Stack Pointer 寄存器的值
A2-A15	A2-A15 通用寄存器的值
EXCCAUSE	保存异常原因
EXCVADDR	保存引发异常的地址
DEPC	触发 Double Exception，保存第二次引发异常的地址
DEBUGCAU	触发 Debug Exception，保存引发异常的原因
CPENABLE	协处理器使能控制寄存器

3.2 栈帧结构

Xtensa 架构的栈帧结构如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

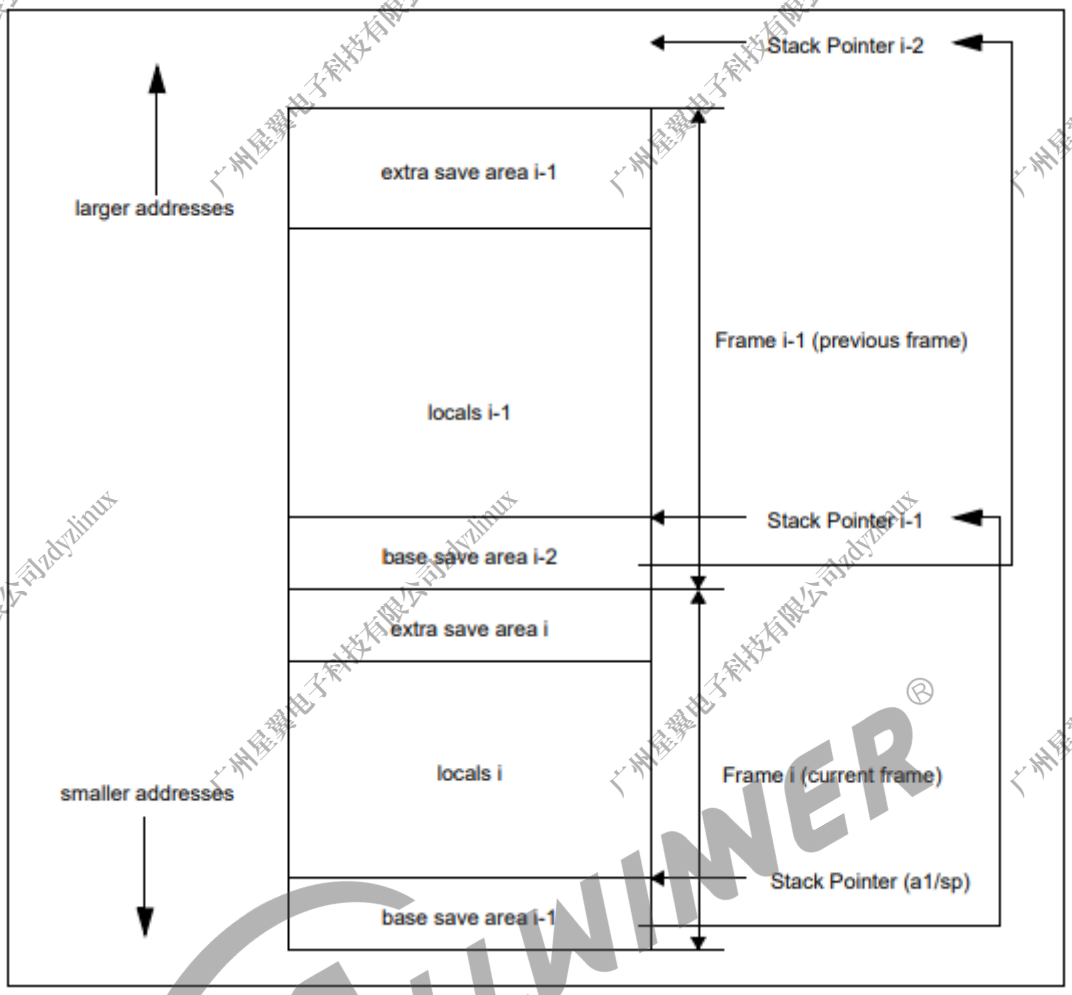

图 3-1: Stack Frame Layout

说明

摘自《Xtensa® Instruction Set Architecture (ISA) Reference Manual》

其中 base save area 就是我们栈回溯需要关心的部分，base save area 保存着调用者的 A0-A3 寄存器，即该部分的长度为 16byte。

3.3 执行工具自动回溯

手动推导栈回溯通常比较困难，所以系统中提供了自动推导栈回溯的命令 backtrace，可以在串口控制台输入 “backtrace [任务名]” 来获取指定任务的调用栈。

系统中拿到的调用栈仍然只是多个地址，还需要通过 addr2line 来获取源码所在的文件及行号，DSP SDK 中封装了 callstack 命令来更方便地完成这一步。

说明

系统崩溃触发的异常中也会自动调用 backtrace 以回溯崩溃现场的调用栈。

接口介绍：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
int backtrace(char *taskname, void *output[], int size, int offset, print_function print_func);

参数:
  taskname  : 任务名字；可为NULL，表示回溯当前任务
  output    : 栈回溯结果保存数组，可以为NULL
  size      : output数组大小，可为0
  offset    : 栈回溯保存结果的偏移，可为0
  print_func : 打印函数，在任务上下文可用printf，中断上下文可用printk

返回值:
  level : 回溯层次
```

步骤 1

在 SDK 编译环境下，开启 backtrace 功能。配置项如下：

```
./build.sh menuconfig
  Components options  --->
    Allwinner components  --->
      Backtrace Support  --->
          [*] Enable backtrace for HIFI4/5   # 使能 backtrace
          [*]   Enable backtrace exccuase str # 在触发异常时打印异常类型字符串
```

步骤 2

终端控制台输入命令，使用 backtrace 命令对指定的任务进行回溯。

> backtrace [taskname]

步骤 3

1. 将回溯信息拷贝，保存在 DSP SDK 的根目录下 xxx.txt 文件。
2. 在加载环境变量后，执行 “callstack xxx.txt” 获取以下回溯信息。

举个例子，相关打印如下：

```
mhd_start_scan at /xxx/mhd_apps_scan.c:334
# mhd_start_scan表示函数名，/xxx/mhd_apps_scan.c表示函数所在的文件路径，334表示函数调用处的行号。

mhd_softap_start at /xxx/mhd_apps_softap.c:263
wifi_recv_cb at /xxx/mhd_api_test.c:624
mhd_get_host_sleep at /xxx/mhd_apps_wifi.c:81
bswap_16 at /xxx/aw-alsa-lib/bswap.h:39
 (inlined by) convert_from_s16 at ??:?
linear_init at /xxx/pcm_rate_linear.c:378
resampler_basic_interpolate_single at /xxx/resample_speexdsp.c:395
__fill_vb2_buffer at /xxx/videobuf2-v4l2.c:392
cci_read at /xxx/cci_helper.c:728
ecdsa_signature_to_asn1 at /xxx/ecdsa.c:294
cmd_wifi_fwlog at /xxx/mhd_api_test.c:449

# 函数调用顺序为从下到上，即 cmd_wifi_fwlog -> ecdsa_signature_to_asn1 -> cci_read ... -> mhd_start_scan
```

说明

callstack 命令依靠固件对应的 elf 文件；如后续修改过代码并重新生成了固件，elf 文件将发生改变，不能保证解析得到的栈回溯信息正确。请确保解析所用的 elf 文件与机器固件版本一致，可预先备份，在回溯的时候先替换 tlf 文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.4 计算寄存器手动回溯

在无法使用系统内置的 backtrace 功能时，可以通过手动推导调用链来获取调用信息，尽管这可能更为复杂。

栈帧分析

首先，查看当前的 PC 和 A0（LR）即可获得最近的两级调用链。如果这些信息足够进行分析，则无需进一步推导。

说明

需要注意的是，LR 指向的是函数返回地址，因此实际的跳转指令是在 LR 之前的那条指令。由于我们获取到的地址未执行，直接使用 LR 进行 addr2line 可能会得到尚未执行到的代码行号。为获得更准确的调用位置（考虑到跳转指令长度为 3 字节），应使用 LR-3 进行 addr2line 查找；后续通过 LR 获取的函数调用位置也同样适用此原则。

接下来，我们查看 A1 寄存器（即 SP）。根据栈帧结构，上一级的 LR (即 LR[i-1]) 位于 SP-16，上一级的 SP (即 SP[i-1]) 位于 SP-12。通过获取新的 LR 和 SP，LR 用于查找函数跳转地址，而 SP 则用于获取更上一级的 LR (即 LR[i-2]) 和 SP (即 SP[i-2])。

下图展示栈帧分析的思路，举例：

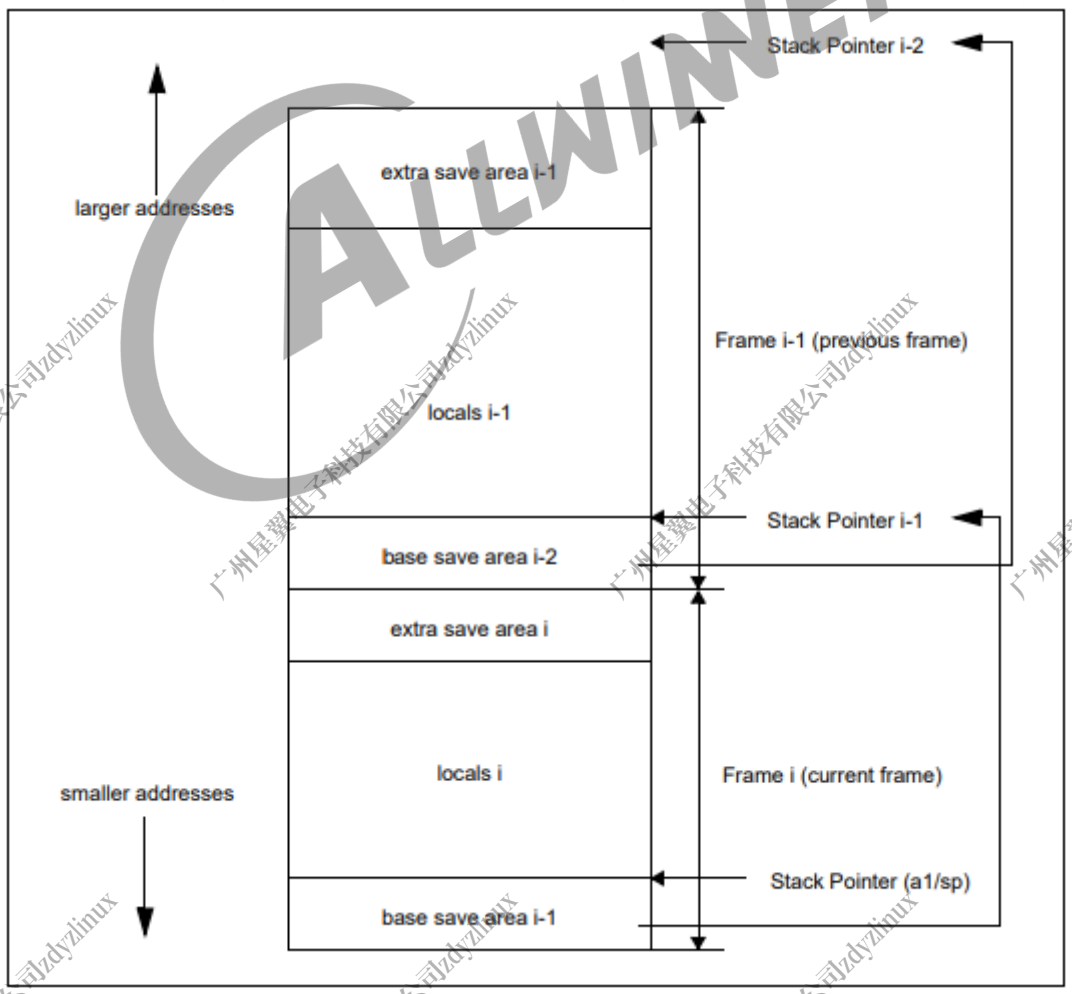

图 3-2: Stack Frame Layout

版权所有 © 珠海全志科技股份有限公司。保留一切权利

可以看到和 backtrace 获取的调用栈一致。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 指令调试

在面对一些复杂的内存越界或逻辑异常问题时，与无关代码的类似行为可能导致固定的断点和观测点误触发。为了解决这个问题，可以借助串口控制台命令或直接嵌入调试代码的方式，实现动态增减断点和观测点的功能。这种灵活的调试方法使得开发者能够根据实际情况调整调试策略，提高了调试的有效性和针对性。

4.1 breakpoint 断点调试

断点调试是指利用 CPU 的硬件或软件断点来进行程序的调试。开发者可以在特定的地址设置断点，当程序执行到该地址时，会触发 Debug Exception 异常。此时，调试器能够捕捉到该异常，并根据异常信息进行分析和排查问题。

通过这种方式，开发者可以迅速判断程序流程是否正确，是否执行到预期的代码位置。这不仅可以帮助有效定位问题，还能深入分析程序的执行状态和变量的值，从而更加高效地进行故障排查和性能优化。

设置接口

```
int gdb_set_hw_break(unsigned long addr, enum gdb_bptype type);
作用
  往地址 addr 设置 type 类型的断点
参数
  addr    : 4 字节对齐的监视地址
  type    : BP_HARDWARE_BREAKPOINT，或者直接填1

返回值
  0 表示设置成功，-1 表示设置失败
```

移除接口

```
int gdb_remove_hw_break(unsigned long addr);
作用
  将在地址 addr 设置的断点移除
参数
  addr    : 4 字节对齐的监视地址

返回值
  0 表示移除成功，-1 表示移除失败
```

模块配置

步骤 1

开启相关配置：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
./build.sh menuconfig
  Components options  --->
    Allwinner components  --->
      Watchpoint Components Support  --->
        [*] DSP RTOS Watchpoint
```

步骤 2

在终端输入对应命令：

> breakpoint [set | remove] addr

```
作用 : 设置程序断点，当前仅使用硬件断点；
  set   : 设置断点；
  remove : 取消断点；
  addr  : 在该地址设置断点。
```

断点异常分析

可以按照崩溃现场分析章节 的内容进行分析;

4.2 watchpoint 观察点调试

观察点调试是指利用 CPU 的硬件观察点（watchpoint）来进行程序调试。开发者可以在特定地址设置观察点，并指定特定操作属性（如读取、写入或执行）。当 CPU 对该地址进行相应的指定操作时，会触发 Debug Exception 异常。

通过捕捉此异常，调试器可以分析当前的执行状态和上下文。这种方法使开发者能够迅速判断某块内存是否被修改、读取或以其他方式访问，从而更有效地排查内存相关的问题，尤其是在调试复杂的逻辑错误、内存越界或数据竞争时，观察点调试提供了重要的支持。

表 4-1: 观察点属性表

属性	作用
write	监视写操作
read	监视读操作
access	监视访问操作，包括读和写

设置接口

```
int gdb_set_hw_watch(unsigned long addr, enum gdb_bptype type);
作用
  往地址 addr 设置 type 类型的观察点
参数
  addr    : 4 字节对齐的监视地址
  type    : BP_WRITE_WATCHPOINT，BP_READ_WATCHPOINT, BP_ACCESS_WATCHPOINT 分别对应写监视、读监视、读
    写监视，或者直接填入 2、3、4。
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
返回值
  0 表示设置成功，-1 表示设置失败
```

移除接口

```
int gdb_remove_hw_watch(unsigned long addr);
作用
  将在地址 addr 设置的观察点移除
参数
  addr    : 4 字节对齐的监视地址

返回值
  0 表示移除成功，-1 表示移除失败
```

模块配置

步骤 1

开启相关配置：

```
./build.sh menuconfig
  Components options  --->
    Allwinner components  --->
      Watchpoint Components Support  --->
        [*] DSP RTOS Watchpoint
```

步骤 2

在终端输入对应命令：

```
watchpoint [write | read | access | remove] addr
```

```
作用 : 设置硬件观察点，当前仅使用硬件断点
  write  : 监视写操作
  read   : 监视读操作
  access : 监视访问操作
  remove : 取消观察点
  addr   : 在该地址设置/取消观察点
```

观察点异常分析

可以按照崩溃现场分析章节的内容进行分析;

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 外部 JTAG 调试接口

使用外部调试接口可以在不修改代码的情况下进行调试。

5.1 Xplorer

根据《DSP_FreeRTOS 系统软件 _ 开发指南》的 “环境准备 -> Windows 调试仿真环境” 的步骤，连接调试器并完成设置后，您可以按照以下步骤进行调试：

1. 连接调试器：确保您的调试器已经正确连接到计算机，并连接到目标 DSP 设备。
2. 打开调试工具：启动您所使用的 IDE（集成开发环境），并确保您的项目已经加载。
3. 配置调试器：在 IDE 中，配置调试器设置，确认选择了正确的调试器及其连接方式。
4. 建立链接：选择连接到目标设备的选项，确保调试器已成功连接到 DSP。
5. 开始调试：在 IDE 中找到并点击 “Debug” 按钮，启动调试会话。这时，您的程序将开始运行在 DSP 上。
6. 暂停程序：一旦程序运行起来，点击 “暂停” 符号，这将停止 DSP 的执行，允许您检查当前状态。
7. 查看调用栈：在调试视图中，您应该能够看到当前的调用栈（call stack）。这通常位于调试窗口的一部分，列出当前调用的函数及其调用关系。

跳转到代码位置：点击调用栈中的任何一项，IDE 应该会自动跳转到相应的代码行，您可以查看当前执行位置的代码并进行进一步的分析。

这样，您就可以在不修改代码的情况下，快速跟踪和排查问题。请确保您按照特定的规范和步骤操作，如有必要，参考文档中的具体设置步骤以确保调试环境的正确配置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

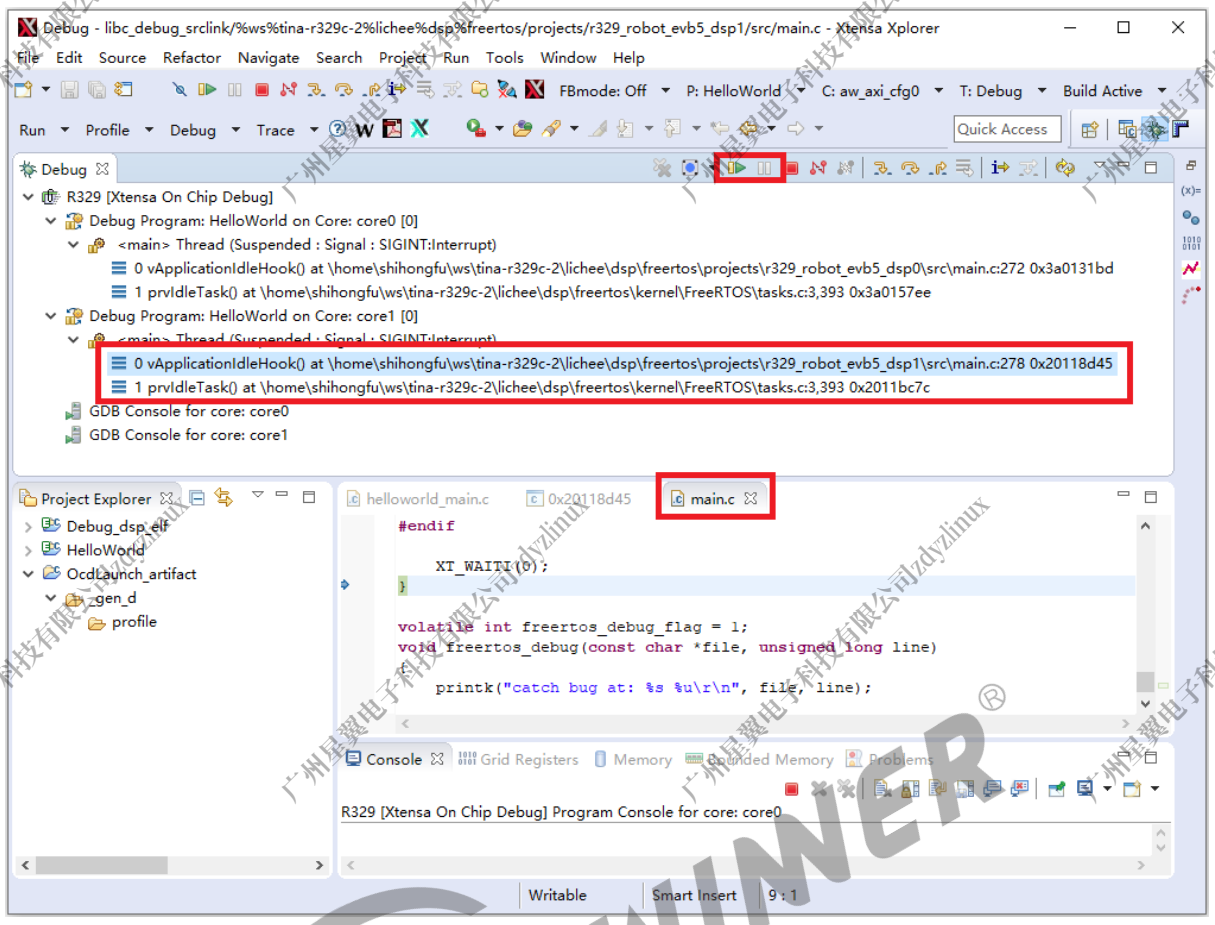

图 5-1: Xplorer 调试 15

按需选择单步执行进行调试：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

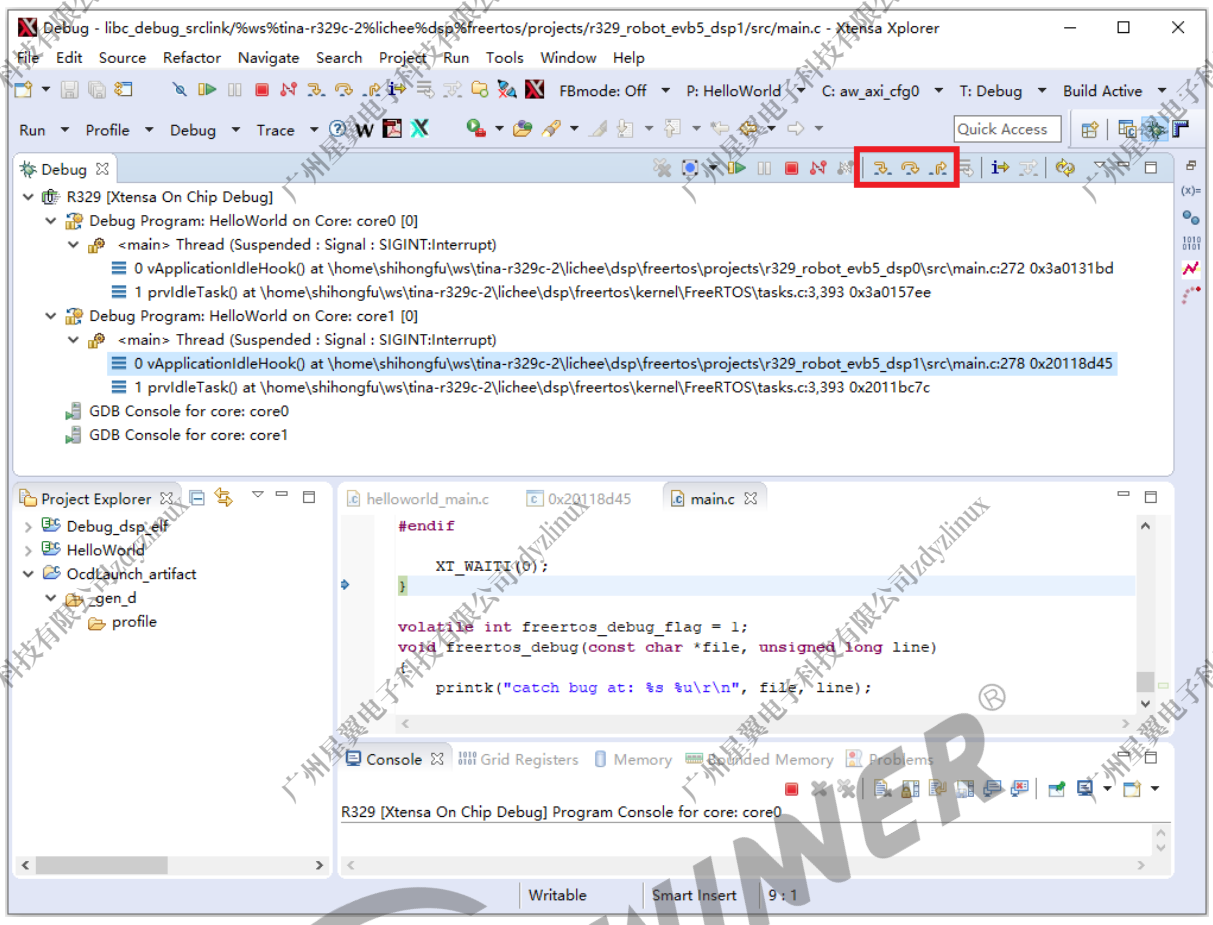

图 5-2: Xplorer 调试 16

5.2 OpenOCD

5.2.1 搭建

步骤 1

OpenOCD 使用的驱动与 SEGGER J-Link 的不同，需要使用 UsbDriverTool 替换驱动：

1. 打开 UsbDriverTool，找到 J-Link driver；
2. 右键 J-Link driver 选择 Install WinUSB；
3. J-Link driver 变为 J-Link driver (WinUSB)，断开调试器再连接；

说明

如需继续使用 SEGGER J-Link，需要还原驱动，即再右键 J-Link driver 选择 Restore default driver。重启 PC 也会还原驱动，如果无法连接，可以检查下驱动是不是已经还原。

获取 UsbDriverTool：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

<https://visualgdb.com/UsbDriverTool/>

步骤 2

修改配置文件，举例：

```
R329配置文件

set CHIPNAME      R329
#set ONLYCPU        1
if { [info exists ONLYCPU] } {

  if { $ONLYCPU == 0 } {
    echo "debug dsp0"
    set CHIPCORES     2
    set CPUTAPID(0)    0x12003017
    set CPUTAPID(1)    0x12003015
    set CORENAME(0)    dsp1
    set CORENAME(1)    dsp0
  } else {
    set CHIPCORES     1
    echo "debug dsp1"
    set CPUTAPID(0)    0x12003017
    set CORENAME(0)    dsp1
  }
} else {
  echo "debug dsp0 & dsp1"
  set CHIPCORES     2
  set CPUTAPID(0)    0x12003017
  set CPUTAPID(1)    0x12003015
  set CORENAME(0)    dsp1
  set CORENAME(1)    dsp0
}

transport select jtag

for { set THISCORE 0 } { $THISCORE < $CHIPCORES } { incr THISCORE 1 } {
  set TAPID $CPUTAPID($THISCORE)
  set TAPTYPE $CORENAME($THISCORE)
  set BASENAME $CHIPNAME

  echo "$THISCORE: eval jtag newtap $BASENAME $TAPTYPE -irlen 5 -expected-id $TAPID -ignore-version"
  eval jtag newtap $BASENAME $TAPTYPE -irlen 5 -expected-id $TAPID -ignore-version
}

for { set THISCORE 0 } { $THISCORE < $CHIPCORES } { incr THISCORE 1 } {

  if { [info exists ONLYCPU] } {
    if { $ONLYCPU == 0 } {
      if { $THISCORE == 0 } {
        continue
      }
    }
  }

  set TAPTYPE $CORENAME($THISCORE)
  set BASENAME $CHIPNAME
  set TAPNAME $BASENAME.$TAPTYPE
  set TARGETNAME $CORENAME($THISCORE)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    echo "$THISCORE: target create $TARGETNAME xtensa -chain-position $TAPNAME"
    target create $TARGETNAME xtensa -chain-position $TAPNAME
}

for { set THISCORE 0 } { $THISCORE < $CHIPCORES } { incr THISCORE 1 } {
    if { [info exists ONLYCPU] } {
        if { $ONLYCPU == 0 } {
            if { $THISCORE == 0 } {
                continue
            }
        }
    }

    set TARGETNAME $CORENAME($THISCORE)

    echo "$THISCORE: $TARGETNAME configure -event reset-assert-post { soft_reset_halt }"
    $TARGETNAME configure -event reset-assert-post { soft_reset_halt }
}

gdb_report_register_access_error enable

adapter speed 400

for { set THISCORE 0 } { $THISCORE < $CHIPCORES } { incr THISCORE 1 } {
    if { [info exists ONLYCPU] } {
        if { $ONLYCPU == 0 } {
            if { $THISCORE == 0 } {
                continue
            }
        }
    }

    set TARGETNAME $CORENAME($THISCORE)

    echo "$THISCORE: $TARGETNAME xtensa"

    $TARGETNAME xtensa xtdef LX
    $TARGETNAME xtensa xtopt    arnum           32
    $TARGETNAME xtensa xtopt    windowed        1

    #  Exception/Interrupt Options
    $TARGETNAME xtensa xtopt    exceptions      1
    $TARGETNAME xtensa xtopt    hipriints       1
    $TARGETNAME xtensa xtopt    intlevels       4
    $TARGETNAME xtensa xtopt    excmlevel       3

    # Memory Options
    $TARGETNAME xtensa xtmem    sram            0x20100000  1900544
    $TARGETNAME xtensa xtmem    dram            0x3a000000  1048576
    #
    $TARGETNAME xtensa xtmem    sram            0x02500000  4096
    # msgbox
    $TARGETNAME xtensa xtmem    sram            0x03008000  4096
    # dsp cfg
    $TARGETNAME xtensa xtmem    sram            0x07050000  4096
    $TARGETNAME xtensa xtmem    sram            0x07051000  4096
    # R_PRCM
    $TARGETNAME xtensa xtmem    sram            0x07010000  65536
    # R_TIMER
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
$TARGETNAME xtensa xtmem    sram          0x07020000  4096

#  Cache Options
$TARGETNAME xtensa xtmem    icache          64  32768   2
$TARGETNAME xtensa xtmem    dcache          64  32768   4   1

#  Debug Options
$TARGETNAME xtensa xtopt    debuglevel       4
$TARGETNAME xtensa xtopt    ibreaknum        2
$TARGETNAME xtensa xtopt    dbreaknum        2

$TARGETNAME xtensa xtregs   176
$TARGETNAME xtensa xtregfmt contiguous       128
$TARGETNAME xtensa xtreg    pc              0x0020
$TARGETNAME xtensa xtreg    ar0             0x0100
$TARGETNAME xtensa xtreg    ar1             0x0101
$TARGETNAME xtensa xtreg    ar2             0x0102
$TARGETNAME xtensa xtreg    ar3             0x0103
$TARGETNAME xtensa xtreg    ar4             0x0104
$TARGETNAME xtensa xtreg    ar5             0x0105
$TARGETNAME xtensa xtreg    ar6             0x0106
$TARGETNAME xtensa xtreg    ar7             0x0107
$TARGETNAME xtensa xtreg    ar8             0x0108
$TARGETNAME xtensa xtreg    ar9             0x0109
$TARGETNAME xtensa xtreg    ar10            0x010a
$TARGETNAME xtensa xtreg    ar11            0x010b
$TARGETNAME xtensa xtreg    ar12            0x010c
$TARGETNAME xtensa xtreg    ar13            0x010d
$TARGETNAME xtensa xtreg    ar14            0x010e
$TARGETNAME xtensa xtreg    ar15            0x010f
$TARGETNAME xtensa xtreg    ar16            0x0110
$TARGETNAME xtensa xtreg    ar17            0x0111
$TARGETNAME xtensa xtreg    ar18            0x0112
$TARGETNAME xtensa xtreg    ar19            0x0113
$TARGETNAME xtensa xtreg    ar20            0x0114
$TARGETNAME xtensa xtreg    ar21            0x0115
$TARGETNAME xtensa xtreg    ar22            0x0116
$TARGETNAME xtensa xtreg    ar23            0x0117
$TARGETNAME xtensa xtreg    ar24            0x0118
$TARGETNAME xtensa xtreg    ar25            0x0119
$TARGETNAME xtensa xtreg    ar26            0x011a
$TARGETNAME xtensa xtreg    ar27            0x011b
$TARGETNAME xtensa xtreg    ar28            0x011c
$TARGETNAME xtensa xtreg    ar29            0x011d
$TARGETNAME xtensa xtreg    ar30            0x011e
$TARGETNAME xtensa xtreg    ar31            0x011f
$TARGETNAME xtensa xtreg    lbeg            0x0200
$TARGETNAME xtensa xtreg    lend            0x0201
$TARGETNAME xtensa xtreg    lcount          0x0202
$TARGETNAME xtensa xtreg    sar             0x0203
$TARGETNAME xtensa xtreg    prefctl         0x0228
$TARGETNAME xtensa xtreg    windowbase      0x0248
$TARGETNAME xtensa xtreg    windowstart     0x0249
$TARGETNAME xtensa xtreg    configid0       0x02b0
$TARGETNAME xtensa xtreg    configid1       0x02d0
$TARGETNAME xtensa xtreg    ps              0x02e6
$TARGETNAME xtensa xtreg    br              0x0204
$TARGETNAME xtensa xtreg    acclo           0x0210
$TARGETNAME xtensa xtreg    acchi           0x0211
$TARGETNAME xtensa xtreg    m0              0x0220
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
$TARGETNAME xtensa xtreg m1 0x0221
$TARGETNAME xtensa xtreg m2 0x0222
$TARGETNAME xtensa xtreg m3 0x0223
$TARGETNAME xtensa xtreg ae_ovf_sar 0x03f0
$TARGETNAME xtensa xtreg ae_bithead 0x03f1
$TARGETNAME xtensa xtreg ae_ts_fts_bu_bp 0x03f2
$TARGETNAME xtensa xtreg ae_cw_sd_no 0x03f3
$TARGETNAME xtensa xtreg ae_cbegin0 0x03f6
$TARGETNAME xtensa xtreg ae_cend0 0x03f7
$TARGETNAME xtensa xtreg ae_cbegin1 0x03f8
$TARGETNAME xtensa xtreg ae_cend1 0x03f9
$TARGETNAME xtensa xtreg aed0 0x1010
$TARGETNAME xtensa xtreg aed1 0x1011
$TARGETNAME xtensa xtreg aed2 0x1012
$TARGETNAME xtensa xtreg aed3 0x1013
$TARGETNAME xtensa xtreg aed4 0x1014
$TARGETNAME xtensa xtreg aed5 0x1015
$TARGETNAME xtensa xtreg aed6 0x1016
$TARGETNAME xtensa xtreg aed7 0x1017
$TARGETNAME xtensa xtreg aed8 0x1018
$TARGETNAME xtensa xtreg aed9 0x1019
$TARGETNAME xtensa xtreg aed10 0x101a
$TARGETNAME xtensa xtreg aed11 0x101b
$TARGETNAME xtensa xtreg aed12 0x101c
$TARGETNAME xtensa xtreg aed13 0x101d
$TARGETNAME xtensa xtreg aed14 0x101e
$TARGETNAME xtensa xtreg aed15 0x101f
$TARGETNAME xtensa xtreg u0 0x1020
$TARGETNAME xtensa xtreg u1 0x1021
$TARGETNAME xtensa xtreg u2 0x1022
$TARGETNAME xtensa xtreg u3 0x1023
$TARGETNAME xtensa xtreg aep0 0x1024
$TARGETNAME xtensa xtreg aep1 0x1025
$TARGETNAME xtensa xtreg aep2 0x1026
$TARGETNAME xtensa xtreg aep3 0x1027
$TARGETNAME xtensa xtreg fcr_fsr 0x1029
$TARGETNAME xtensa xtreg ibreakenable 0x0260
$TARGETNAME xtensa xtreg memctl 0x0261
$TARGETNAME xtensa xtreg ddr 0x0268
$TARGETNAME xtensa xtreg ibreaka0 0x0280
$TARGETNAME xtensa xtreg ibreaka1 0x0281
$TARGETNAME xtensa xtreg dbreaka0 0x0290
$TARGETNAME xtensa xtreg dbreaka1 0x0291
$TARGETNAME xtensa xtreg dbreakc0 0x02a0
$TARGETNAME xtensa xtreg dbreakc1 0x02a1
$TARGETNAME xtensa xtreg epc1 0x02b1
$TARGETNAME xtensa xtreg epc2 0x02b2
$TARGETNAME xtensa xtreg epc3 0x02b3
$TARGETNAME xtensa xtreg epc4 0x02b4
$TARGETNAME xtensa xtreg epc5 0x02b5
$TARGETNAME xtensa xtreg depc 0x02c0
$TARGETNAME xtensa xtreg eps2 0x02c2
$TARGETNAME xtensa xtreg eps3 0x02c3
$TARGETNAME xtensa xtreg eps4 0x02c4
$TARGETNAME xtensa xtreg eps5 0x02c5
$TARGETNAME xtensa xtreg excsave1 0x02d1
$TARGETNAME xtensa xtreg excsave2 0x02d2
$TARGETNAME xtensa xtreg excsave3 0x02d3
$TARGETNAME xtensa xtreg excsave4 0x02d4
$TARGETNAME xtensa xtreg excsave5 0x02d5
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
$TARGETNAME xtensa xtreg cpenable 0x02e0
$TARGETNAME xtensa xtreg interrupt 0x02e2
$TARGETNAME xtensa xtreg intset 0x02e2
$TARGETNAME xtensa xtreg intclear 0x02e3
$TARGETNAME xtensa xtreg intenable 0x02e4
$TARGETNAME xtensa xtreg vecbase 0x02e7
$TARGETNAME xtensa xtreg exccause 0x02e8
$TARGETNAME xtensa xtreg debugcause 0x02e9
$TARGETNAME xtensa xtreg ccount 0x02ea
$TARGETNAME xtensa xtreg prid 0x02eb
$TARGETNAME xtensa xtreg icount 0x02ec
$TARGETNAME xtensa xtreg icountlevel 0x02ed
$TARGETNAME xtensa xtreg excvaddr 0x02ee
$TARGETNAME xtensa xtreg ccompare0 0x02f0
$TARGETNAME xtensa xtreg ccompare1 0x02f1
$TARGETNAME xtensa xtreg a0 0x0000
$TARGETNAME xtensa xtreg a1 0x0001
$TARGETNAME xtensa xtreg a2 0x0002
$TARGETNAME xtensa xtreg a3 0x0003
$TARGETNAME xtensa xtreg a4 0x0004
$TARGETNAME xtensa xtreg a5 0x0005
$TARGETNAME xtensa xtreg a6 0x0006
$TARGETNAME xtensa xtreg a7 0x0007
$TARGETNAME xtensa xtreg a8 0x0008
$TARGETNAME xtensa xtreg a9 0x0009
$TARGETNAME xtensa xtreg a10 0x000a
$TARGETNAME xtensa xtreg a11 0x000b
$TARGETNAME xtensa xtreg a12 0x000c
$TARGETNAME xtensa xtreg a13 0x000d
$TARGETNAME xtensa xtreg a14 0x000e
$TARGETNAME xtensa xtreg a15 0x000f
$TARGETNAME xtensa xtreg b0 0x0010
$TARGETNAME xtensa xtreg b1 0x0011
$TARGETNAME xtensa xtreg b2 0x0012
$TARGETNAME xtensa xtreg b3 0x0013
$TARGETNAME xtensa xtreg b4 0x0014
$TARGETNAME xtensa xtreg b5 0x0015
$TARGETNAME xtensa xtreg b6 0x0016
$TARGETNAME xtensa xtreg b7 0x0017
$TARGETNAME xtensa xtreg b8 0x0018
$TARGETNAME xtensa xtreg b9 0x0019
$TARGETNAME xtensa xtreg b10 0x001a
$TARGETNAME xtensa xtreg b11 0x001b
$TARGETNAME xtensa xtreg b12 0x001c
$TARGETNAME xtensa xtreg b13 0x001d
$TARGETNAME xtensa xtreg b14 0x001e
$TARGETNAME xtensa xtreg b15 0x001f
$TARGETNAME xtensa xtreg psintlevel 0x2006
$TARGETNAME xtensa xtreg psum 0x2007
$TARGETNAME xtensa xtreg pswoe 0x2008
$TARGETNAME xtensa xtreg psexcm 0x2009
$TARGETNAME xtensa xtreg pscallinc 0x200a
$TARGETNAME xtensa xtreg psowb 0x200b
$TARGETNAME xtensa xtreg acc 0x200c
$TARGETNAME xtensa xtreg dbnum 0x2011
$TARGETNAME xtensa xtreg ae_overflow 0x2014
$TARGETNAME xtensa xtreg ae_sar 0x2015
$TARGETNAME xtensa xtreg ae_cwrap 0x2016
$TARGETNAME xtensa xtreg ae_bitptr 0x2017
$TARGETNAME xtensa xtreg ae_bitsused 0x2018
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    $TARGETNAME xtensa xtreg   ae_tablesize     0x2019
    $TARGETNAME xtensa xtreg   ae_first_ts      0x201a
    $TARGETNAME xtensa xtreg   ae_nextoffset    0x201b
    $TARGETNAME xtensa xtreg   ae_searchdone    0x201c
    $TARGETNAME xtensa xtreg   roundmode        0x201d
    $TARGETNAME xtensa xtreg   invalidflag      0x201e
    $TARGETNAME xtensa xtreg   divzeroflag      0x201f
    $TARGETNAME xtensa xtreg   overflowflag     0x2020
    $TARGETNAME xtensa xtreg   underflowflag    0x2021
}

targets
```

将配置文件命名为 “xtensa-core-aw-r329.cfg” 放到该目录：

```
<openocd>/share/openocd/scripts/target
```

根据实际情况调整调试器访问的内存区域，以确保正确读取和监控相关数据。

```
# Memory Options
    $TARGETNAME xtensa xtmem   sram        0x20100000  1900544 // dsp1的运行地址及长度
    $TARGETNAME xtensa xtmem   dram        0x3a000000  1048576 // dsp0的运行地址及长度
    # msgbox
    $TARGETNAME xtensa xtmem   sram        0x03008000  4096 // 外设地址及长度
```

其他搭载 HiFi4 平台可以参照该配置，修改以下变量使用：

```
set CHIPNAME      R528 // 自定义
set ONLYCPU       1
set CPUTAPID(0)   0x12003017 // 可以使用Xplorer进行探测
set CORENAME(0)   dsp1
```

5.2.2 常用命令

命令	作用
targets	列出所有调试目标
targets [TargetName]	选择一个调试目标
halt	停止运行
step	单步指令执行，需要目标已停止运行
resume	恢复运行，需要目标已停止运行
reg	列出所有寄存器的值，需要目标已停止运行
reg [name]	列出指定寄存器的值，需要目标已停止运行
read_memory [address] [width] [count]	以指定宽度、数量读取指定地址内存
write_memory [address] [width] [data]	以指定宽度向指定地址内存写入值

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 操作演示

为了捕捉错误的锁操作，在锁的实现中添加调试代码，当出现异常时死循环等待连接调试器：

```c
static inline int shared_unlock(struct access_lock_t *alock)
{
    int shared_cnt = -1;
    unsigned long flags;

    flags = hal_spin_lock_irqsave(&alock->lock);
    if (alock->shared_cnt > 0)
        shared_cnt = --alock->shared_cnt;
    hal_spin_unlock_irqrestore(&alock->lock, flags);

    if (shared_cnt == 0)
        hal_sem_post(alock->sem);
#if 1
    if (shared_cnt < 0) {
        volatile int loop = 1;
        printf("%s BUG!!!\n", __func__);
        flags = hal_spin_lock_irqsave(&alock->lock);
        while (loop);
        hal_spin_unlock_irqrestore(&alock->lock, flags);
    }
#endif
```

图 6-1: shared_unlock 调试代码

同时添加错误的使用代码：

lichee > dsp > freertos > components > thirdparty > openamp > sunxi_helper > C memory.c

```c
static int _shmem_metal_io_block_write(struct metal_io_region *io, unsigned long offset,
        const void *restrict src, memory_order order, int len)
{
    int ret;
    unsigned long flags = standby_memory_lock(g_alock, io->mem_flags);

    ret = shmem_metal_io_block_write(io, offset, src, order, len);

    standby_memory_unlock(g_alock, flags);
    standby_memory_unlock(g_alock, flags);

    return ret;
}
```

图 6-2: shared_unlock2 调试代码

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6.1 OpenOCD

1. 进入 “<openocd>/bin” 目录。
2. 启动 OpenOCD：运行命令：“openocd.exe -f interface/jlink.cfg -f target/xtensa-core-aw-r329.cfg”
3. 如果出现报错，提示 “Error: couldn’t bind tcl to socket on port 6666: No error”，这意味着您需要修改 TCL socket 的端口。可以在启动 OpenOCD 时添加参数 “-p <new_port>” 来指定一个新的端口。
4. 如果没有错误信息，则说明启动成功，日志信息如下：

```
Info : Listening on port 7777 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : J-Link V9 compiled May  7 2021 16:26:12
Info : Hardware version: 9.70
Info : VTarget = 3.287 V
Info : clock speed 400 kHz
Info : JTAG tap: R329.dsp1 tap/device found: 0x12003017 (mfg: 0x00b (Intersil), part: 0x2003, ver: 0x1)
Info : JTAG tap: R329.dsp0 tap/device found: 0x12003015 (mfg: 0x00a (I.T.T.), part: 0x2003, ver: 0x1)
Info : starting gdb server for dsp1 on 3333
Info : Listening on port 3333 for gdb connections
Info : starting gdb server for dsp0 on 3334
Info : Listening on port 3334 for gdb connections
Info : [dsp1] Debug controller was reset.
Info : [dsp1] Core was reset.
Info : [dsp0] Debug controller was reset.
Info : [dsp0] Core was reset.
```

5. 使用 Telnet 协议连接 OpenOCD，输入命令：telnet localhost 4444。
6. 输入 “targets” 命令可以查看当前的调试目标列表，其中带 “*” 的选项表示当前选择的调试目标：

```
> targets
   TargetName       Type       Endian TapName            State
--  ------------------ ---------- ------ ------------------ ------------
 0  dsp1             xtensa     little R329.dsp1          running
 1* dsp0             xtensa     little R329.dsp0          running
```

7. 输入命令 targets dsp1 以选择 dsp1 作为当前调试目标;
8. 当需要进行调试时，输入命令 halt 来暂停 dsp1。随后，输入 reg 可以查看所有寄存器的状态（这些寄存器根据配置文件进行配置）；

```
> halt
> reg
===== Xtensa registers
(0) pc (/32): 0x20130657
(1) ar0 (/32): 0x00000000
(2) ar1 (/32): 0x00000000
(3) ar2 (/32): 0x00000000
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
(4) ar3 (/32): 0x00060423
...
```

9. 运用栈帧分析的方法获取调用栈地址，您可以按照以下步骤进行操作：

    1. 首先输入命令 reg pc 来获取程序计数器（PC）的当前值，并记录下该值。
    2. 然后输入 reg a0 和 reg a1 来获取这两个寄存器的值。
    3. 将 a0 的高 2 位置为 0。接下来判断这个新值是否位于 DSP 的运行地址范围内：
        - 如果是，则将这个值视为返回地址（Link Register, LR），并计算 LR - 3 来记录这个新值。
        - 如果不是，则结束获取地址的过程。
    4. 接着判断 a1 是否位于 DSP 的运行地址范围内：
        - 如果是，则将这个值视为栈指针（sp）。
        - 如果不是，则结束获取地址。
    5. 计算新的栈指针值，即 sp - 0x10，并使用命令 read_memory [address] 32 2 读取内存。然后返回第二步继续获取下一级调用栈地址。

请注意，在进行以上操作时，需要确保按照具体的 DSP 运行环境和手册中的规定进行调整，确保地址和数据格式正确无误。

```
> targets
   TargetName       Type       Endian TapName            State
--  ------------------ ---------- ------ ------------------ ------------
 0  dsp1             xtensa     little R329.dsp1          running
 1* dsp0             xtensa     little R329.dsp0          running

> targets dsp1
> halt
> reg pc
pc (/32): 0x20130657
> reg a0
a0 (/32): 0xa0130a1f
> reg a1
a1 (/32): 0x20158170
> read_memory 0x20158160 32 2
0xa012e07f 0x201581a0
> read_memory 0x20158190 32 2
0xa012f13b 0x201581c0
> read_memory 0x201581b0 32 2
0xa012f229 0x201581e0
> read_memory 0x201581d0 32 2
0xa01471e0 0x20158220
> read_memory 0x20158210 32 2
0xa012ec79 0x20158250
> read_memory 0x20158240 32 2
0xa012ee2a 0x20158270
> read_memory 0x20158260 32 2
0xa01328ed 0x201582c0
> read_memory 0x201582b0 32 2
0xa0133336 0x201582f0
> read_memory 0x201582e0 32 2
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
0xa0133fd0 0x20158330
> read_memory 0x20158320 32 2
0xa012c172 0x20158360
> read_memory 0x20158350 32 2
0xa012c0c5 0x20158380
> read_memory 0x20158370 32 2
0xa012c0c5 0x201583a0
> read_memory 0x20158390 32 2
0xa012c0c5 0x201583c0
> read_memory 0x201583b0 32 2
0xa012c0c5 0x201583e0
> read_memory 0x201583d0 32 2
0xa012c0c5 0x20158400
> read_memory 0x201583f0 32 2
0xa012cf8b 0x20158420
> read_memory 0x20158410 32 2
0x0 0x20158440
```

10. 将所有记录的地址保存为 bk.txt 文件后，您可以通过以下步骤获取问题现场的调用栈：
11. 确保您已经将之前记录的所有调用栈地址写入到 bk.txt 文件中。
12. 输入命令 callstack bk.txt 以直接依据 bk.txt 中的地址获取调用栈信息。或者，您也可以使用以下命令：

    ```
    ./backtrace.sh bk.txt xxx/dsp.elf
    ```

    在该命令中，xxx/dsp.elf 需要替换为您的实际 ELF 文件路径，这样可以利用脚本来处理文件并获取调用栈信息。

确保您的配置正确，文件路径有效，并且您有执行该命令的权限。

```
0x20130657 standby_memory_unlock <root>/include/access_lock.h:111
0x20130a1c _shmem_metal_io_block_write <root>/components/thirdparty/openamp/sunxi_helper/memory.c:310
0x2012e07c metal_io_block_write <root>/components/thirdparty/openamp/libmetal/lib/io.c:85
0x2012f138 rpmsg_virtio_send_offchannel_nocopy <root>/components/thirdparty/openamp/open-amp/lib/rpmsg/
    rpmsg_virtio.c:397
0x2012f226 rpmsg_virtio_send_offchannel_raw <root>/components/thirdparty/openamp/open-amp/lib/rpmsg/
    rpmsg_virtio.c:475
0x201471dd rpmsg_send_offchannel_raw <root>/components/thirdparty/openamp/open-amp/lib/rpmsg/rpmsg.c:124
0x2012ec76 rpmsg_send_ns_message <root>/components/thirdparty/openamp/open-amp/lib/rpmsg/rpmsg.c:138
0x2012ee27 rpmsg_create_ept <root>/components/thirdparty/openamp/open-amp/lib/rpmsg/rpmsg.c:303
0x201328ea openamp_ept_open <root>/components/thirdparty/openamp/rpmsg_ctrl/rpmsg_master.c:81
0x20133333 rpmsg_ctrldev_create <root>/components/thirdparty/openamp/rpmsg_ctrl/rpmsg_master.c:610
0x20133fcd common_rpmsg_master_probe <root>/components/thirdparty/openamp/rpmsg_ctrl/rpmsg_master_driver
    .c:17
0x2012c16f dev_probe_internal <root>/components/aw/device/device.c:283
0x2012c0c2 dev_probe_internal <root>/components/aw/device/device.c:516
0x2012c0c2 dev_probe_internal <root>/components/aw/device/device.c:516
0x2012c0c2 dev_probe_internal <root>/components/aw/device/device.c:516
0x2012c0c2 dev_probe_internal <root>/components/aw/device/device.c:516
0x2012c0c2 dev_probe_internal <root>/components/aw/device/device.c:516
0x2012cf88 dev_mgr_thread <root>/components/aw/device/device.c:1128
```

分析崩溃现场

通过 OpenOCD 修改指令，手动使其崩溃：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
> halt
> reg pc
pc (/32): 0x2013065c
> read_memory 0x2013065c 32 1
0xadff7a56
> write_memory 0x2013065c 32 0xadffffff
> read_memory 0x2013065c 32 1
> resume
```

获取崩溃现场日志：

```
----------------------------------------
----------------------------------------
Core  0 register dump:
PC    : 0x20130667  PS    : 0x00060633  A0    : 0xa0130a1f  A1    : 0x20158170
A2    : 0x20158c40  A3    : 0xffffffff  A4    : 0x201581f0  A5    : 0x00000000
A6    : 0x00000010  A7    : 0x20114070  A8    : 0xa013064e  A9    : 0x00000001
A10   : 0x00000001  A11   : 0x00060620  A12   : 0x00000000  A13   : 0x00000000
A14   : 0x1b040000  A15   : 0xfffffe4f  SAR   : 0x00000008  EXCCAUSE: 0x00000000
EXCVADDR: 0x00000000  LBEG  : 0x201480b2  LEND  : 0x201480d9  LCOUNT : 0xffffffff

------------
0x20130667
0x20130A1F
0x2012E07F
0x2012F13B
0x2012F229
0x201471E0
0x2012EC79
0x2012EE2A
0x201328ED
0x20133336
0x20133FD0
0x2012C172
0x2012C0C5
0x2012C0C5
0x2012C0C5
0x2012C0C5
0x2012C0C5
0x2012CF8B
------------

Stack Data:
  （略）
0x20158160: 0xa012e07f  0x201581a0  0x00000010  0x201b34c4
0x20158170: 0x00000001  0x201581b0  0x20158c40  0x00000000
0x20158180: 0x201581f0  0x00000000  0x00000010  0x20114070
0x20158190: 0xa012f13b  0x201581c0  0x2015a8d0  0x00000000
0x201581a0: 0x201581e0  0x00000005  0x00000010  0x00000001
0x201581b0: 0xa012f229  0x201581e0  0x2015b16c  0x2015a8d0
0x201581c0: 0x201581e0  0x00000010  0x00000028  0x00000001
0x201581d0: 0xa01471e0  0x20158220  0x2015b16c  0x00000400
0x201581e0: 0x00000400  0x00000035  0x00000000  0x00000028
0x201581f0: 0xa01471e0  0x20158220  0x2015b16c  0x00000400
0x20158200: 0x1b040000  0x1b040010  0x00000028  0x00000000
0x20158210: 0xa012ec79  0x20158250  0x2012f1d0  0x00000400
0x20158220: 0x000001f0  0x2015a8d0  0x2015bba8  0x000000ff
0x20158230: 0x00000035  0x20158270  0x00000028  0x1b040010
0x20158240: 0xa012ee2a  0x20158270  0x2015c0fc  0x00000000
0x20158250: 0x00000035  0x20158270  0x00000028  0x00000001
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
0x20158260: 0xa01328ed 0x201582c0 0x2015c0fc 0x2015b16c
0x20158270: 0x786e7573 0x70722c69 0x5f67736d 0x6c727463
0x20158280: 0x00000000 0x00000000 0x00000000 0x00000000
0x20158290: 0x00000400 0x00000000 0x2015c0fc 0x2015b16c
0x201582a0: 0x00000073 0x00000001 0xffffffff 0xff000000
0x201582b0: 0xa0133336 0x201582f0 0x2010fbb4 0x2015c0fc
0x201582c0: 0xa0133336 0xffffffff 0x2010fbb4 0x00000000
0x201582d0: 0x00000000 0x00000400 0x2015b1c4 0x20132a7c
0x201582e0: 0xa0133fd0 0x20158330 0x20114410 0x201b34d8
0x201582f0: 0x20132bdc 0x2015bd60 0x20101ba0 0xfffffff0
0x20158300: 0xffffffff 0xffffffff 0x20114410 0x201b34d8
0x20158310: 0x20114310 0x2015b16c 0x201b34d8 0x20132a7c
0x20158320: 0xa012c172 0x20158360 0x20149788 0x2010b3e0
0x20158330: 0x20132bdc 0x00000000 0x00000004 0x2015b2a0
0x20158340: 0x201033a0 0x0000000d 0x2012beb1 0x00000000
0x20158350: 0xa012c0c5 0x20158380 0x20149788 0x2014944c
0x20158360: 0x00000000 0x00000001 0x00000000 0x2015a8d0
0x20158370: 0xa012c0c5 0x201583a0 0x2014944c 0x201494c0
0x20158380: 0xffffffed 0x00000001 0x2012beb1 0x55534544
0x20158390: 0xa012c0c5 0x201583c0 0x201494c0 0x20149530
0x201583a0: 0x00000000 0x00000000 0x430ce000 0x00000000
0x201583b0: 0xa012c0c5 0x201583e0 0x20149530 0x20149588
0x201583c0: 0x00000000 0x00000000 0x00000001 0x201142c0
0x201583d0: 0xa012c0c5 0x20158400 0x20149588 0x201496b8
0x201583e0: 0x00000000 0x00000000 0x2012beb1 0x00000000
0x201583f0: 0xa012cf8b 0x20158420 0x201496b8 0x00000000
0x20158400: 0x00000000 0x00000000 0x201b0148 0x201b2070
0x20158410: 0x00000000 0x20158440 0x00000000 0x00000000
（略）
```

分析流程：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
Core  0 register dump:
PC      : 0x20130667  PS      : 0x00060633  A0      : 0xa0130a1f  A1      : 0x20158170
A2      : 0x20158c40  A3      : 0xffffffff  A4      : 0x201581f0  A5      : 0x00000000
A6      : 0x00000010  A7      : 0x20114070  A8      : 0xa013064e  A9      : 0x00000001
A10     : 0x00000001  A11     : 0x00060620  A12     : 0x00000000  A13     : 0x00000000
A14     : 0x1b040000  A15     : 0xffffffe4  SAR     : 0x00000008  EXCCAUSE: 0x00000000
EXCVADDR: 0x00000000  LBEG    : 0x201480b2  LEND    : 0x201480d9  LCOUNT  : 0xffffffff

------------
（略）
------------

Stack Data:
（略）
0x20158160: 0xa012e07f  0x201581a0  0x00000010  0x201b34c4
0x20158170: 0x00000001  0x201581b0  0x20158c40  0x00000000
0x20158180: 0x201581f0  0x00000000  0x00000010  0x20114070
0x20158190: 0xa012f13b  0x201581c0  0x2015a8d0  0x00000000
0x201581a0: 0x201581e0  0x00000005  0x00000010  0x00000001
0x201581b0: 0xa012f229  0x201581e0  0x2015b16c  0x2015a8d0
0x201581c0: 0x201581e0  0x00000010  0x00000028  0x00000001
0x201581d0: 0xa01471e0  0x20158220  0x2015b16c  0x00000400
0x201581e0: 0x00000400  0x00000035  0x00000000  0x00000028
0x201581f0: 0xa01471e0  0x20158220  0x2015b16c  0x00000400
0x20158200: 0x1b040000  0x1b040010  0x00000028  0x00000000
0x20158210: 0xa012ec79  0x20158250  0x2012f1d0  0x00000400
0x20158220: 0x000001f0  0x2015a8d0  0x2015bba8  0x000000ff
0x20158230: 0x00000035  0x20158270  0x00000028  0x1b040010
0x20158240: 0xa012ee2a  0x20158270  0x2015c0fc  0x00000000
0x20158250: 0x00000035  0x20158270  0x00000028  0x00000001
0x20158260: 0xa01328ed  0x201582c0  0x2015c0fc  0x2015b16c
0x20158270: 0x786e7573  0x70722c69  0x5f67736d  0x6c727463
0x20158280: 0x00000000  0x00000000  0x00000000  0x00000000
0x20158290: 0x00000400  0x00000000  0x2015c0fc  0x2015b16c
0x201582a0: 0x00000073  0x00000001  0xffffffff  0xff000000
0x201582b0: 0xa0133336  0x201582f0  0x2010fbb4  0x2015c0fc
0x201582c0: 0xa0133336  0xffffffff  0x2010fbb4  0x00000000
0x201582d0: 0x00000000  0x00000400  0x2015b1c4  0x20132a7c
0x201582e0: 0xa0133fd0  0x20158330  0x20114410  0x201b34d8
0x201582f0: 0x20132bdc  0x2015bd60  0x20101ba0  0xfffffff0
0x20158300: 0xffffffff  0xffffffff  0x20114410  0x201b34d8
0x20158310: 0x20114310  0x2015b16c  0x201b34d8  0x20132a7c
0x20158320: 0xa012c172  0x20158360  0x20149788  0x2010b3e0
```

0x20130667
0x20130a1c
0x2012e07c
0x2012f138
0x2012f226
0x201471dd
0x2012ec76
0x2012ee27
0x201328ea
0x20133333
0x20133fcd
0x2012c16f
0x2012c0c2
0x2012c0c2
0x2012c0c2
0x2012c0c2
0x2012c0c2
0x2012cf88

图 6-3: 崩溃现场分析示例

使用 callstack bk.txt 或./backtrace.sh bk.txt xxx/dsp.elf 获取一致的崩溃现场调用栈信息，确保与 backtrace 和 OpenOCD 栈回溯所提供的地址匹配。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。